Ensemble Methods

Project – PAC-Bayesian Learning of Weighted Majority Votes

M2 Computer Science – MALIA

Guillaume Metzler
Institut de Communication (ICOM)
Université de Lyon, Université Lumière Lyon 2
Laboratoire ERIC UR 3083, Lyon, France

guillaume.metzler@univ-lyon2.fr

Abstract. The goal of this project is to study, implement and compare learning algorithms for weighted majority votes that come with PAC-Bayesian risk certificates. You will build a small library, run a rigorous experimental study on the datasets of the course, and explore one research direction based on a recent paper. The project mobilizes Sections 3 to 7 of the lecture notes (in particular the self-bounding part, Sections 6–7) and notebooks 3 to 5.

Groups of two persons are allowed to do this project. The work shall be submitted at the following mail address

guillaume.metzler@univ-lyon2.fr

before the deadline mentioned in your program. The submission shall contain (i) the Python code (package or notebooks) used to produce *all* the results of the report, (ii) a report in PDF (10 to 15 pages, figures and tables included) presenting the methods, the experimental protocol, the results and their analysis. The code shall reproduce the results presented in the report.

1 Datasets

You will use the datasets of the course (`datasets.zip`, loaded with `prepdata.py`), with **at least 15 datasets**. As in the first project, they can be divided into two groups (balanced / imbalanced), which may require different performance measures. In addition, you will use `make_moons` and `make_classification` from `scikit-learn` for illustrations.

2 Part A – A library for PAC-Bayesian majority votes (mandatory)

Starting from `pbtools.py` (you may rewrite it entirely), implement a Python class `PBMajorityVote` with a `scikit-learn`-like interface (`fit`, `predict`, `certificate`) which:

1. builds a set of n voters by bagging (decision trees, with a parameter for the size of the bootstrap samples and for the randomization of the trees) and keeps track of the out-of-bag samples;
2. computes the out-of-bag statistics: individual losses, tandem losses and disagreements, with the sizes n_1 and n_2 of the smallest OOB sets and intersections;
3. learns a posterior ρ among: uniform, minimizer of the first-order PAC-Bayes-λ bound (Thiemann et al., 2017), minimizer of the tandem bound (Masegosa et al., 2020), minimizer of a PAC-Bayesian C-bound (Germain et al., 2015; Viallard et al., 2021);
4. returns the certificates (first-order, tandem and C-bound) of the learned vote, computed with the kl inversion, for a confidence δ given by the user.

The code must be tested: check your kl inversion against a brute-force search, and check the identity $R(G_\rho) = e_\rho + \frac{1}{2} d_\rho$ on a test sample. Explain in the report how the out-of-bag construction makes the certificates valid (Theorem 7.1 of the lecture notes).

3 Part B – Experimental study (mandatory)

Protocol. For each dataset, use at least 5 random train/test splits (stratified). Hyperparameters that are tuned (depth of the trees, number of features per split, ...) must be tuned by cross-validation on the training set only. Report means and standard deviations, and use a statistical test (e.g. Wilcoxon signed-rank test over the datasets) when you compare two methods.

Methods to compare.

- a random forest (`scikit-learn`) and AdaBoost, as baselines without certificates;
- your `PBMajorityVote` with the four posteriors (uniform, FO, TND, C-bound);
- a single decision tree and a linear SVM, to measure the benefit of the vote.

Expected results and questions.

1. A table of the test risks (and of a balanced measure for the imbalanced group) and a table of the certificates. Which method gives the best vote? the best certificate? Are the certificates non-vacuous (smaller than 0.5) on all datasets?
2. The influence of the number of voters $n \in \{10, 50, 100, 200, 500\}$ and of the bootstrap size on the test risk and on the certificates (use figures).
3. The influence of the diversity: for a few datasets, plot the Gibbs risk, the disagreement, the risk of the vote and the bounds as functions of the number of features tried at each split. Discuss the strength/diversity trade-off in the light of the C-bound.
4. The computation times of the different methods.

4 Part C – A research direction (choose one)

C1 – Boosting with certificates. Learn the voters with AdaBoost or gradient boosting on one part of the data and learn their weights by minimizing the tandem bound or the C-bound on the other part. Compare with the weights given by the boosting algorithm, in terms of risk, certificate and margins (Schapire et al., 1998; Roy et al., 2016).

C2 – Kernels as priors. Following Letarte et al. (2019) (and the first project of the course), use random Fourier features as voters with the kernel as prior. Learn a pseudo-posterior over the features (Gibbs posterior, or minimization of a second-order bound), and compare with an RBF SVM and with the gradient boosting with random Fourier features of the first project.

C3 – Tighter bounds for the vote. Read Wu et al. (2021) and implement the Chebyshev–Cantelli bound with the PAC-Bayes-Bennett inequality (CCPBB). Compare it with the tandem bound and the C-bound, as certificates and as learning objectives.

C4 – Linear classifiers and data-dependent priors. Implement PBGD (Germain et al., 2009) with Gaussian posteriors, with a prior centred at $\mathbf{0}$ and with a data-dependent prior (Ambroladze et al., 2006; Parrado-Hernández et al., 2012). Compare the certificates and the risks with those of a linear SVM, as a function of the proportion of data used to learn the prior.

C5 – Stochastic majority votes. Read Zantedeschi et al. (2021) and implement their Dirichlet posterior over the weights of the vote on a small number of voters (e.g. 20 trees). Compare the certificates with those of Part B.

For the chosen direction, the report must contain a short presentation of the method (with the key equations and their justification), your implementation choices, and an experimental comparison on at least 5 datasets.

5 Evaluation

Part A: correctness and quality of the code, tests	5 points
Part B: protocol, results, figures and analysis	8 points
Part C: understanding of the method, implementation, experiments	5 points
Quality of the report (clarity, notation, references)	2 points

Suggestions for the report

Introduction. Present the context (ensemble methods, need for guarantees) and the objectives.

Methodology. Introduce the notation, then the PAC-Bayesian bounds you use and the learning algorithms, with the key equations: explain what is minimized and why the certificate is valid. Present the performance measures and the experimental protocol.

Results. Tables and figures, always with a sentence explaining what they show. Compare methods that are comparable, and discuss both the risk *and* the tightness of the certificates.

Conclusion. Summarize your findings, their limits and possible extensions.

References

Ambroladze, A., Parrado-Hernández, E., and Shawe-Taylor, J. (2006). Tighter PAC-Bayes bounds. In *Advances in Neural Information Processing Systems (NIPS)*, volume 19, pages 9–16.

Germain, P., Lacasse, A., Laviolette, F., and Marchand, M. (2009). PAC-Bayesian learning of linear classifiers. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pages 353–360.

Germain, P., Lacasse, A., Laviolette, F., Marchand, M., and Roy, J.-F. (2015). Risk bounds for the majority vote: From a PAC-Bayesian analysis to a learning algorithm. *Journal of Machine Learning Research*, 16:787–860.

Letarte, G., Morvant, E., and Germain, P. (2019). Pseudo-Bayesian learning with kernel Fourier transform as prior. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 89 of *PMLR*, pages 768–776.

Masegosa, A. R., Lorenzen, S. S., Igel, C., and Seldin, Y. (2020). Second order PAC-Bayesian bounds for the weighted majority vote. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33.

Parrado-Hernández, E., Ambroladze, A., Shawe-Taylor, J., and Sun, S. (2012). PAC-Bayes bounds with data dependent priors. *Journal of Machine Learning Research*, 13:3507–3531.

Roy, J.-F., Marchand, M., and Laviolette, F. (2016). A column generation bound minimization approach with PAC-Bayesian generalization guarantees. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 51 of *PMLR*, pages 1241–1249.

Schapire, R. E., Freund, Y., Bartlett, P., and Lee, W. S. (1998). Boosting the margin: A new explanation for the effectiveness of voting methods. *The Annals of Statistics*, 26(5):1651–1686.

Thiemann, N., Igel, C., Wintenberger, O., and Seldin, Y. (2017). A strongly quasiconvex PAC-Bayesian bound. In *Proceedings of the 28th International Conference on Algorithmic Learning Theory (ALT)*, volume 76 of *PMLR*, pages 466–492.

Viallard, P., Germain, P., Habrard, A., and Morvant, E. (2021). Self-bounding majority vote learning algorithms by the direct minimization of a tight PAC-Bayesian C-bound. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, volume 12976 of *LNCS*, pages 167–183. Springer.

Wu, Y.-S., Masegosa, A. R., Lorenzen, S. S., Igel, C., and Seldin, Y. (2021). Chebyshev-Cantelli PAC-Bayes-Bennett inequality for the weighted majority vote. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34.

Zantedeschi, V., Viallard, P., Morvant, E., Emonet, R., Habrard, A., Germain, P., and Guedj, B. (2021). Learning stochastic majority votes by minimizing a PAC-Bayes generalization bound. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34.